

实验二

基于BP神经网络的自整定PID控制仿真

一、实验目的

1. 熟悉神经网络的特征、结构及学习算法。
2. 通过实验掌握神经网络自整定 PID 的工作原理。
3. 了解神经网络的结构对控制效果的影响。
4. 掌握用 MATLAB 实现神经网络控制系统仿真的方法。

二、实验设备及条件

1. 计算机系统。
2. MATLAB 仿真软件。

三、实验原理

在工业控制中, PID 控制是工业控制中最常用的方法。这是因为 PID 控制器结构简单, 实现简单, 控制效果良好, 已得到广泛应用。但是, PID 具有一定的局限性: 被控制对象参数随时间变化时, 控制器的参数难以自动调整以适应外界环境的变化。为了使控制器具有较好的自适应性, 实现控制器参数的自动调整, 可以采用神经网络控制的方法。利用人工神经网络的自学习这一特性, 并结合传统的 PID 控制理论, 构造神经网络 PID 控制器, 实现控制器参数的自动调整。

基于 BP 神经网络的 PID 控制器结构如图 4 所示。控制器由两部分组成: 一是常规 PID 控制器, 用以直接对对象进行闭环控制, 且 3 个参数在线整定; 二是神经网络 NN, 根据系统的运行状态, 学习调整权系数, 从而调整 PID 参数, 达到某种性能指标的最优化。

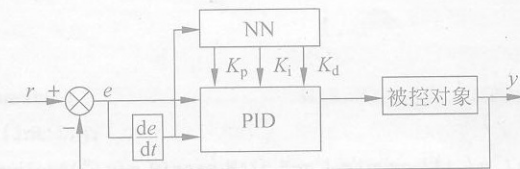


图 4 基于神经网络的 PID 控制器结构

图 4 中神经网络采用结构为 4-5-3 型的 BP 网络。BP 网络是一种单向传播的多层前向网络。输入节点对应系统的运行状态量,如系统的偏差与偏差变化率,必要时要进行归一化处理。输入变量的个数取决于被控系统的复杂程度,输出节点对应的是 PID 的 3 个可调参数。由于输出不能为负,所以输出层活化函数取非负的 Sigmoid 函数,隐含层取正负对称的 Sigmoid 函数。本系统选取的 BP 网络结构如图 5 所示。

网络的学习过程由正向和反向传播两部分组成。如果输出层不能得到期望输出,那么转入反向传播过程,通过修改各层神经元的权值,使得输出误差信号最小。

输出层节点分别对应 3 个可调参数 K_p 、 K_i 、 K_d 。

取性能指标函数为

$$E(k) = \frac{1}{2} (r(k) - y(k))^2$$

按照梯度下降法修正网络的权系数,即按 $E(k)$ 对加权系数的负梯度方向搜索调整,并附加一使快速收敛全局极小的惯性项

$$\Delta w_{ii}^{(3)}(k) = -\eta \frac{\partial E(k)}{\partial w_{ii}^{(3)}} + \alpha \Delta w_{ii}^{(3)}(k-1)$$

式中, η 为学习速率; α 为惯性系数。

$$\frac{\partial E(k)}{\partial w_{ii}^{(3)}(k)} = \frac{\partial E(k)}{\partial y(k)} \cdot \frac{\partial y(k)}{\partial u(k)} \cdot \frac{\partial u(k)}{\partial o_i^{(3)}} \cdot \frac{\partial o_i^{(3)}}{\partial net_i^{(3)}} \cdot \frac{\partial net_i^{(3)}(k)}{\partial w_{ii}^{(3)}(k)}$$

又

$$\begin{cases} \frac{\partial net_i^{(3)}}{\partial w_{ii}^{(3)}} = o_i^{(2)}(k) \\ \frac{\partial E(k)}{\partial y(k)} = -e(k) \end{cases}$$

其中, $u(k)$ 为控制器在 k 时刻的输出; $o_i^{(3)}$ 为输出层各节点的输出, $o_1^{(3)} = K_p$, $o_2^{(3)} = K_i$, $o_3^{(3)} = K_d$; $net_i^{(3)}$ 为输出层各节点的输入。

若采用增量式数字 PID 控制算法,则有

$$\frac{\partial u(k)}{\partial o_1^{(3)}} = e(k) - e(k-1)$$

$$\frac{\partial u(k)}{\partial o_2^{(3)}} = e(k)$$

$$\frac{\partial u(k)}{\partial o_3^{(3)}} = e(k) - 2e(k-1) + e(k-2)$$

四、实验步骤

(1) 被控对象为一时变非线性对象,数学模型可表示为

$$y(k) = \frac{a(k)y(k-1)}{1 + y^2(k-1)} + u(k-1)$$

式中系数 $a(k)$ 是慢时变的, $a(k) = 1.2(1 - 0.8e^{-0.1k})$ 。

(2) 如图 5 所示确定 BP 网络的结构,选 4-5-3 型的 BP 网络,各层加权系数的初值取

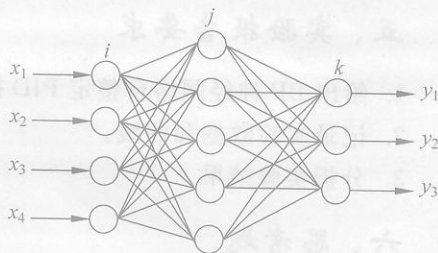


图 5 BP 网络结构

区间 $[-0.5, 0.5]$ 上的随机数,选定学习速率 $\eta=0.25$ 和惯性系数 $\alpha=0.05$ 。

(3) 在 MATLAB 下依据整定原理编写仿真程序并调试。

(4) 给定输入为阶跃信号,运行程序,记录实验数据和控制曲线。

(5) 修改神经网络参数,如学习速率、隐含层神经元个数等,重复步骤(4)。

(6) 分析数据和控制曲线。

五、实验报告要求

1. 给出 BP 神经网络自整定 PID 控制系统的设计过程和程序清单。

2. 记录实验数据和曲线。

3. 分析实验结果。

六、思考题

1. BP 神经网络的学习速度和什么因素有关?

2. BP 神经网络的收敛结果是否全局最优?

附录 仿真程序

```
clear all;
close all;
xite = 0.25;           % 惯性系数
alfa = 0.02;           % 学习率
IN = 4;                % 输入层的神经元个数
H = 10;                % 隐含层的神经元个数
Out = 3;               % 输出层的神经元个数
```

```
wi = [ 0.4634   -0.4173   0.3190   0.4563
       0.1839   0.3021   0.1112   0.3395
      -0.3182   0.0470   0.0850  -0.0722
      -0.6266   0.0846   0.3751  -0.6900
      -0.3224   0.1440  -0.2873  -0.0193
      -0.0232  -0.0992   0.2636   0.2011
      -0.4502  -0.2928   0.0062  -0.5640
      -0.1975  -0.1332   0.1981   0.0422
       0.0521   0.0673  -0.5546  -0.4830
      -0.6016  -0.4097   0.0338  -0.1503];
```

% 初始化隐含层权值

```
wi_1 = wi; wi_2 = wi; wi_3 = wi;
```

```
% wo = [ -0.5071   0.0114  -0.3941  -0.1591  -0.3959;
%         0.0510  -0.1721   0.0243  -0.1671   0.0813;
%         -0.3386   0.2869  -0.3036   0.0701   0.2293];
```

```
% wo = [0.7576   0.2616   0.5820  -0.1416  -0.1325;
```

```

%      - 0.1146    0.2949    0.8352    0.2205    0.4508;
%      0.7201    0.4566    0.7672    0.4962    0.3632];

wo = [- 0.1620    0.3674    0.1959
      - 0.0337   - 0.1563   - 0.1454
        0.0898    0.7239    0.7605
        0.3349    0.7683    0.4714
        0.0215    0.5896    0.7143
       - 0.0914    0.4666    0.0771
        0.4270    0.2436    0.7026
        0.0215    0.4400    0.1121
        0.2566    0.2486    0.4857
        0.0198    0.4970    0.6450]';

%      wo = 0.50 * randn(Out,H);          % 初始化输出层权值
wo_1 = wo; wo_2 = wo; wo_3 = wo;

x = [0,0,0];          % 定义数组并初始化

u_1 = 0; u_2 = 0; u_3 = 0; u_4 = 0; u_5 = 0;

y_1 = 0; y_2 = 0; y_3 = 0;          % 初始化对象输出

oh = zeros(H,1);
I = oh;
error_2 = 0;
error_1 = 0;

ts = 0.001;          % 采样周期
for k = 1:1:6000
time(k) = k * ts;
rin(k) = 1;          % 参考输入

a(k) = 1.2 * (1 - 0.8 * exp(- 0.1 * k));          % 系数 a(k)
yout(k) = a(k) * y_1 / (1 + y_1^2) + u_1;          % 对象输出差分方程
% yout(k) = exp(- 1/8) * y_1 + 0.16 * (1 - exp(- 1/8)) * u_9;
% + 0.16 * (exp(- 0.5) - exp(- 1)) * u_3;
error(k) = rin(k) - yout(k);          % 误差
xi = [rin(k), yout(k), error(k), 1];          % 神经网络输入
x(1) = error(k) - error_1;
x(2) = error(k);
x(3) = error(k) - 2 * error_1 + error_2;
epid = [x(1); x(2); x(3)];
I = xi * wi';          % 隐含层输入
for j = 1:H

```

```

    oh(j) = (exp(I(j)) - exp(-I(j)))/(exp(I(j)) + exp(-I(j)));
end
K = wo * oh; % 输出层输入
for l = 1:Out;
    K(l) = exp(K(l))/(exp(K(l)) + exp(-K(l))); % 输出层输出
end
Kp(k) = K(1);Ki(k) = K(2);Kd(k) = K(3); % 输出层输出的 PID 3 个系数
Kpid = [Kp(k),Ki(k),Kd(k)];

du(k) = Kpid * epid;

u(k) = u_1 + du(k);
if u(k) >= 10;
    u(k) = 10; % 限制输出
end
if u(k) <= -10
    u(k) = -10;
end

dyu(k) = sgn((yout(k) - y_1)/(u(k) - u_1 + 0.0000001));

% 输出层权值整定
for j = 1:Out;
    dk(j) = 2/(exp(K(j)) + exp(-K(j)))^2;
end
for l = 1:Out
    delta3(l) = error(k) * dyu(k) * epid(l) * dk(l);
end

for l = 1:Out
    for i = 1:H
        d_wo = xite * delta3(l) * oh(i) + alfa * (wo_1 - wo_2);
    end
end
wo = wo_1 + d_wo;

% 隐含层权值整定
for i = 1:H
    d0(i) = 4/(exp(I(i)) + exp(-I(i)))^2;
end
segma = delta3 * wo;
for i = 1:H
    delta2(i) = d0(i) * segma(i);
end

```

```
d_wi = xite * delta2' * xi + alfa * (wi_1 - wi_2);
wi = wi_1 + d_wi;
```

```
% 参数更新
```

```
u_5 = u_4; u_4 = u_3; u_3 = u_2; u_2 = u_1; u_1 = u(k);
```

```
y_2 = y_1; y_1 = yout(k);
```

```
wo_3 = wo_2;
```

```
wo_2 = wo_1;
```

```
wo_1 = wo;
```

```
wi_3 = wi_2;
```

```
wi_2 = wi_1;
```

```
wi_1 = wi;
```

```
error_2 = error_1;
```

```
error_1 = error(k);
```

```
end
```

```
% 绘图
```

```
figure(1)
```

```
plot(time, rin, 'r', time, yout, 'b');
```

```
xlabel('time');
```

```
ylabel('rin, yout');
```

```
figure(2)
```

```
plot(time, error, 'r');
```

```
xlabel('time(s)');
```

```
ylabel('error');
```

```
figure(3)
```

```
plot(time, u, 'r');
```

```
xlabel('time(s)');
```

```
ylabel('u');
```

```
figure(4)
```

```
subplot(311);
```

```
plot(time, Kp, 'r');
```

```
xlabel('time(s)');
```

```
ylabel('Kp');
```

```
subplot(312);
```

```
plot(time, Ki, 'g');
```

```
xlabel('time(s)');
```

```
ylabel('Ki');
```

```
subplot(313);
```

```
plot(time, Kd, 'b');
```

```
xlabel('time(s)');
```

```
ylabel('Kd');
```